is less than 1b model good for llm prompt intent evaluating for safety checking?

Yes, a sub-1B parameter model can be highly effective for LLM prompt intent evaluation and safety checking, provided it is implemented correctly. While it will struggle with open-ended text generation, a sub-1B model functions exceptionally well as a lightweight, fast, and cost-effective **token-level classification guardrail**. arXiv +1

Recent architectural breakthroughs, such as **Qwen 2.5 Instruct 0.5B** or **Llama 3.2 1B**, have proven to be robust feature extractors when fine-tuned or paired with a classification head. arXiv +2

Why Sub-1B Models Work for Safety Checking

- **Low Latency**: Safety evaluation must happen *before* the main LLM processes a prompt. Sub-1B models add minimal overhead (often single-digit milliseconds), preserving a snappy user experience.
- **Cost Efficiency**: Running a 70B model or an external API for every single user input is financially unsustainable at production scale. Sub-1B models can run locally on cheap hardware or edge devices.
- **Narrow Task Scope**: Categorizing intent (e.g., classifying a prompt as `Safe`, `Malicious`, `Hate Speech`, or `Injection`) requires pattern matching rather than complex reasoning. arXiv +4

- **High Sensitivity to Prompt Artifacts**: Small models can be easily confused by apologetic language, polite phrasing, or distracting filler words. OpenReview +4

Best Practices for Using a Sub-1B Safety Evaluator

1. Do Not Use Zero-Shot Prompting

Instead of asking the raw model `Is this prompt safe? Answer Yes or No`, use **Parameter-Efficient Fine-Tuning (PEFT/LoRA)** or train a linear classifier on top of the model's intermediate hidden layers. Research shows the middle layers of these tiny models serve as excellent feature extractors for safety classification. arXiv +1

2. Pair with Heuristic Rules

Combine the sub-1B model with regex patterns, blocklists, and vector database lookups for known injection strings. This covers basic keyword exploits, leaving the model to catch semantic intents. YouTube · AI Anytime +1

3. Use a Tiered Defense Architecture

Deploy the sub-1B model as a front-line filter. If the tiny model outputs high confidence (e.g., $> 95\%$ safe or $> 95\%$ unsafe), act immediately. If the confidence score falls into an ambiguous gray area (e.g., $40\%–70\%$), route the prompt to a larger, more robust safety model like a 5B–8B model (e.g., Meta's Llama-Guard or IBM's Granite Guardian). arXiv

---

If you want to test this out, let me know:

Is there any light weight pretrained or already finetuned model to return safety score for prompting queries?

Yes, several highly optimized, industry-standard lightweight models are specifically designed to return safety evaluations and scores for prompting queries.

Depending on your architecture, you can choose between generative **1B parameter LLM safeguards** (which output structured safety categories or scores) or ultra-fast, encoder-based **DeBERTa/BERT classifiers** (which natively return

layout. It outputs `safe` or `unsafe`, followed by specific hazard categories (e.g., cyberattacks, hate speech).

- **The Ultra-Lightweight Option**: Meta officially optimized this into Llama Guard 3-1B-INT4. It is heavily quantized, takes up only **440MB of space**, and can run at high speeds directly on consumer CPUs and mobile devices.
- **Extracting a Score**: While it natively outputs text tokens (`safe` / `unsafe`), you can extract a continuous confidence score by looking at the model's **output log probabilities (logprobs)** for the `unsafe` token versus the `safe` token.

AI at Meta +4

2. Meta's Llama Prompt Guard 86M

If a 1B model is still too large, Meta also offers Llama Prompt Guard, an incredibly tiny **86-million-parameter model** based on a BERT-like architecture.

Llama +1

- **How it works**: It is explicitly trained to detect two things: **Injections** (jailbreaks/malicious intent) and **Jailbreaks** (string manipulations).
- **Output**: It functions as a sequence classifier. It returns a direct tensor containing **probability scores** mapping out exactly how likely a query is to be a prompt injection attempt.

Reddit · r/LocalLLaMA

3. Fine-Tuned DeBERTa-v3 Models (Hugging Face)

For strict mathematical scoring (0.0 to 1.0) with zero LLM overhead, the community heavily relies on **DeBERTa-v3-base** (around 80M–100M active parameters) fine-tuned on content moderation datasets. DeBERTa outperforms standard BERT variants significantly due to its disentangled attention mechanism.

Show all

- **Examples on Hugging Face**: Models like `protectai/deberta-v3-base-prompt-`

Use code with caution.

Hugging Face

Comparison Matrix

Model Name	Size / Parameters	Primary Target	Output Type	Best Use Case
Llama Prompt Guard 86M	~86M (~340 MB)	Injection & Jailbreak	Continuous Probability Score	Front-line high-speed firewall
DeBERTa-v3-base Finetunes	~100M (~400 MB)	Generalized Toxicity/Safety	Continuous Probability Score	Extreme low-latency pipelines
Llama Guard 3 1B INT4	~1B (440 MB quantized)	Complex Intent / 16+ Categories	Text Token (extractable Logprobs)	Detailed enterprise risk auditing

If you are leaning toward one of these, let me know:

- Do you prefer a **Python snippet** to see how to load and pull the probability score from **Prompt Guard 86M**, or would you prefer a setup for **Llama Guard 1B**?

If I train from raw dataset, does it take long for training? for example for 86m model?

No, training an 86M parameter model from scratch or fine-tuning it does not take long. Because 86M is incredibly small by modern AI standards, it can easily be trained on standard consumer hardware or free cloud GPUs (like Google Colab) in a matter of minutes to a few hours.

However, the exact time depends entirely on whether you are doing **Pretraining** (learning language from scratch) or **Fine-Tuning** (teaching an existing model to detect safety).

Scenario A: Fine-Tuning a Pretrained 86M Model (Highly

Scenario B: Pretraining an 86M Model from Scratch

If by "raw dataset" you mean initializing an empty model with random weights and training it to understand grammar, vocabulary, *and* safety intent all at once, it takes longer because you need billions of tokens.

Assuming a dataset of **1 billion tokens** (roughly 3GB of raw text):

- **On a single A100 GPU**: **~8 to 12 hours**.
- **On a standard consumer GPU (RTX 3090/4090)**: **~18 to 24 hours**.

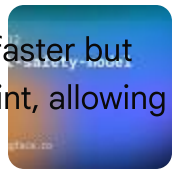
What Actually Dictates Your Training Time?

To estimate your exact training duration, look at these three variables:

14 sites

1. **Sequence Length**: Long inputs exponentially slow down training. If your prompts are short (e.g., under 128 tokens), training will be lightning fast. If you feed the model entire 2,000-word essays, training will take 4x to 8x longer.

SalKhan12/prompt-safety-model - Hugging Face
Jul 19, 2025 — This model is a fine-tuned version of bert-base-uncased on an [illegible]



2. **Batch Size**: Larger batch sizes (like 32 or 64) process data much faster but require more GPU VRAM. An 86M model has a tiny memory footprint, allowing you to use huge batch sizes even on cheap 16GB GPUs.

Hugging Face

3. **Data Cleanliness**: Pre-tokenizing your dataset and saving it to disk before starting the training loop prevents your CPU from bottlenecking your GPU.

Download Llama
Llama 3.3: 70B. Text. Llama 3.3: 70B. • Multilingual open source large language model. • Experience 405B performance and quality a...

Practical Recommendation

Llama

is there any other free huggingface model like llamaguard 3b or 86m

Yes, Hugging Face hosts several excellent free, open-source safety models that range from tiny, specialized encoder classifiers (similar to Meta's 86M Prompt Guard) to powerful, lightweight generative guard models under 3B parameters.

The best lightweight alternatives categorized by their architecture and size include:

Generative Safeguard Models (Sub-3B LLMs)

These models act like text-generation models but are explicitly fine-tuned to evaluate a prompt and output safety tags or risk categories.

- **ShieldGemma 2B (by Google)**: Built on top of the Gemma-2 architecture, ShieldGemma 2B is specifically optimized to predict whether a user prompt violates safety policies. It evaluates inputs against four key harm categories: hate speech, harassment, sexually explicit content, and dangerous content.
- **Granite Guardian 3.2 3B-A800M (by IBM)**: IBM offers the highly optimized

millisecond latency and tiny memory usage, these models use similar text classification heads to output standard probability scores.

- **PromptShield (by sumitranjan)**: Built on top of `xlm-roberta-base`, the PromptShield model is a highly lightweight option tuned to catch complex prompt injections and adversarial string attacks. It achieves near-perfect classification scores on jailbreak validation sets.
- **Toxic Prompt RoBERTa (by daniel-de-leon)**: A 125M parameter model (`daniel-de-leon/toxic-prompt-roberta`) that punches far above its weight class. In public content moderation benchmarks, it frequently beats large-scale models in baseline toxicity and adversarial prompt detection due to its hyper-focused classification layer.
- **Safe Guard Classifier (by xTRam1)**: An ultra-compact **44M parameter model** explicitly trained to protect LLMs against malicious prompt injections. It is twice as small as Meta's Prompt Guard 86M, making it one of the absolute fastest open-weights firewalls available. Hugging Face +3

Quick Selection Guide

- Choose **ShieldGemma 2B** if you need high accuracy on nuanced, multi-paragraph prompts.
- Choose **Granite Guardian 3B** if you need a model that can also detect system issues like hallucinations or tool-calling exploits.
- Choose **PromptShield** or **Toxic Prompt RoBERTa** if you are deploying to a strict low-latency environment or a CPU-only server. Hugging Face +3

Would you like a **Python pipeline snippet** showing how to quickly run one of these